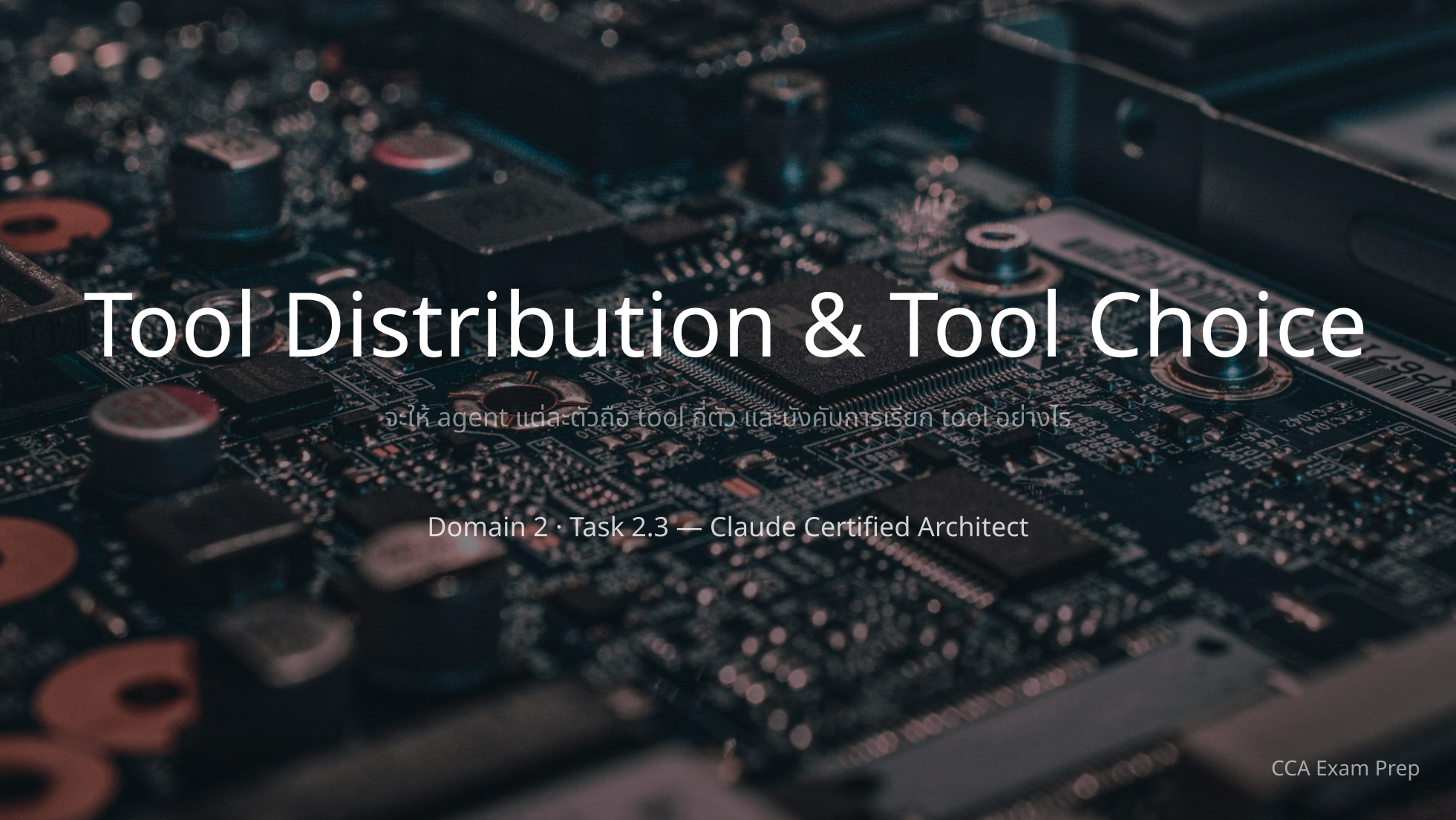




# Tool Distribution & Tool Choice

จะให้ agent แต่ละตัวถือ tool ที่ตัว และยังคำนึงการเรียก tool อย่างไร

Domain 2 · Task 2.3 — Claude Certified Architect

# Tool Distribution คืออะไร?

- คือการตัดสินใจว่าจะแจก tool ตัวไหนให้ **agent ตัวไหนถือ** และถือ **กี่ตัว**
- ตัวเลขหลักในข้อสอบ: optimal คือ **4-5 tools ต่อ agent** และต้อง **scoped strictly** ตามบทบาทของ agent ตัวนั้น

ประโยคที่ต้องจำ

*"The number of tools you give an agent directly affects how reliably it selects the right one."*

ยิ่ง tool เยอะ → **selection accuracy ยิ่งตก**

# The Tool Overload Problem

★ ออกสอบบ่อย

## 🔑 Key Concept

การยัด tool มากเกินไปทำให้ **selection accuracy ลดลง** — ไม่ใช่ทำให้ tool เยอะแล้วโมเดลจะเก่งขึ้น กลับกันคือ**แย่ลง**

### × synthesis agent ไม่ควรมี web search

มันจะไปค้นซ้ำกับข้อมูลที่ถูกส่งมาให้แล้ว (duplicate work)

### × web search agent ไม่ควรมี document analysis tool

คนละบทบาท — เปิดช่อง cross-role misuse

หลักการ: **จำกัด tool ให้อยู่ในบทบาท** เพื่อกัน cross-role misuse และทำให้ agent ปลอดภัย

# tool\_choice Parameter: 3 โหมด

`tool_choice` คือ parameter ที่ควบคุมว่าโมเดลจะเรียก tool อย่างไร  
— มี 3 โหมดที่ต้องแยกให้ออก →

`auto` (ค่า default)

โมเดลตัดสินใจเองว่าจะเรียก tool หรือจะตอบเป็น  
ข้อความธรรมดา

→ ใช้กับงานที่ยืดหยุ่น (flexible interactions)

`any`

โมเดล**ต้องเรียก tool "สักตัวหนึ่ง"** แต่เลือกเองว่าตัว  
ไหน

→ ใช้การรับตี structured output จาก extraction pipeline ที่มี  
หลาย schema

# tool\_choice: โหมด Forced Selection

## Forced selection

โมเดลต้องเรียก tool "ตัวที่ระบุชื่อไว้เจาะจง" เท่านั้น

→ ใช้บังคับขั้นตอนที่เป็น mandatory workflow เช่น ต้องรัน `extract_metadata` ให้เสร็จก่อน แล้วค่อยไปรัน enrichment

## 📌 หลักวิสัย

ถ้าโจทย์ต้องการ **structured output** แน่ๆ → **อย่าใช้ auto**

auto เปิดช่องให้โมเดลตอบเป็นข้อความเปล่า ๆ ได้

`any` — เมื่อ tool ไหนก็ได้

`forced` — เมื่อต้องเป็น tool ตัวใดตัวหนึ่งเจาะจง

# Scoped Cross-Role Tools Pattern

## ⚠️ ปัญหา

ถ้า route ทุก request ผ่าน `coordinator` จะเพิ่ม **2-3 round trips** และ latency ราว **40%** ต่องาน

## ✅ วิธีแก้

ให้ agent ถือ "เวอร์ชันจำกัดสิทธิ์" ของ capability ที่มันใช้บ่อย

เช่น synthesis agent มี scoped `verify_fact` tool ไว้ทำ simple lookup เอง — ส่วน verification ที่ซับซ้อนจริง ๆ ค่อย route ผ่าน coordinator

🔑 เก็บ common case ไว้ทำเอง เพื่อเลี่ยง round-trip latency

# Least Privilege Tool Design

## 🔑 Key Concept

ออกแบบ tool ตามหลัก **least privilege** — แทน generic tool ด้วยเวอร์ชันที่ถูกจำกัดขอบเขต

### ✗ generic

`fetch_url` — ดึง URL อะไรก็ได้

### ✓ constrained

`load_document` — validate เฉพาะ document URL เท่านั้น

## ประโยชน์สองชั้น

🛡️ กัน misuse — agent เอา tool ไปทำอย่างอื่นไม่ได้

🎯 clarifies purpose — จุดประสงค์ของ tool ชัดเจนขึ้น

# Case Study: synthesis agent วิ่งกลับ coordinator บ่อยเกิน

## อาการ

synthesis agent เรียกกลับมา coordinator บ่อยมากเพื่อขอ verify fact ง่าย ๆ → เพิ่ม **2-3 round trips** ต่องาน และ latency พุ่งขึ้น **40%**

## สาเหตุ

การ verify **ทั้งหมด** ถูก route ผ่าน coordinator ทั้งที่ผลวิเคราะห์บอกว่า **85% ของ verification เป็นแค่ simple lookup**

→ เอา common case ไปเดินทางไกลโดยไม่จำเป็น

## ✅ วิธีแก้

ให้ synthesis agent ถือ scoped `verify_fact` tool ไว้ทำ simple lookup เอง แล้ว route เฉพาะ verification ที่ซับซ้อนผ่าน coordinator

# ! Exam Traps ที่ต้องระวัง

1. ใช้ `tool_choice: 'auto'` ทั้งที่โจทย์ต้องการ **structured output**

× ต้องใช้ `any` หรือ forced selection

2. ให้ **agent** ตัวเดียวถือ **18+ tools** แล้วคาดหวังว่าจะเลือกถูกอย่างเชื่อถือได้

× selection accuracy จะพัง — optimal คือ 4-5

3. ใช้ **generic** `fetch_url` แทน **constrained** `load_document`

× เปิดช่องให้ misuse

4. **route simple verification** ทั้งหมดผ่าน **coordinator** ทั้งที่ 85% เป็น **simple lookup**

× เพิ่ม latency โดยไม่จำเป็น — ควรใช้ `scoped cross-role tool`



# Practice Scenario

ลองคิดก่อนดูเฉลย

**โจทย์:** synthesis agent วิ่งกลับมาหา coordinator บ่อย ๆ เพื่อ verify fact ง่าย ๆ เพิ่ม 2-3 round trips ต่องาน latency สูงขึ้น 40% วิเคราะห์แล้วพบว่า **85% ของ verification เป็น simple lookup** — ควรแก้อย่างไร?

A. เพิ่ม parallelism ของ coordinator ให้ประมวลผล verification ได้เร็วขึ้น

B. ตัด fact verification ออกจาก synthesis workflow ไปเลย

C. Cache ผล verification ทั้งหมดไว้ที่ระดับ coordinator

D. ให้ synthesis agent ถือ scoped `verify_fact` tool ไว้ทำ simple lookup เอง แล้ว route เฉพาะ verification ที่ซับซ้อนผ่าน coordinator



หยุดคิดสัก 30 วินาที แล้วค่อยไปสไลด์ถัดไปเพื่อดูเฉลย



## เฉลย Practice Scenario

### คำตอบที่ถูก — Option D

ให้ `scoped verify_fact` tool กับ synthesis agent เพื่อจัดการ **85% ที่เป็น simple lookup** ในตัวเอง เหลือแค่ เคสซับซ้อนที่ route ผ่าน coordinator

→ ตรงกับ **Scoped Cross-Role Tools Pattern** คือการจัด round-trip ของ common case

**A ผิด** — แก้ที่ความเร็วของ coordinator แต่ต้นเหตุคือ round trip ที่ไม่จำเป็น ไม่ใช่ coordinator ช้า

**B ผิด** — ตัด verification กึ่งทำลายความถูกต้องของงาน

**C ผิด** — ยัง route ทุกอย่างผ่าน coordinator อยู่ดี round-trip latency ไม่หาย



# Build Exercise

ฝึกออกแบบตามโจทย์ข้อสอบ →

- 1 ออกแบบ agent 3 บทบาท (web search, document analysis, synthesis) ให้แต่ละตัวมี **scoped tools**
- 2 เพิ่ม scoped `verify_fact` tool ให้ synthesis agent สำหรับ simple lookup
- 3 ตั้ง `tool_choice` แบบ forced selection ที่ document analysis agent ให้ `extract_metadata` รับก่อนเสมอ
- 4 แทน generic `fetch_url` ด้วย constrained `load_document` ที่ validate เฉพาะ document URL
- 5 ทดสอบด้วย query ที่ต้องใช้ครบทั้ง 3 agent แล้วตรวจสอบว่าไม่มี cross-role tool misuse เกิดขึ้น



## Exam Sim — ข้อ 1/5 (ลองคิดก่อนดูเฉลย)

synthesis agent วิ่งกลับมา coordinator บ่อย ๆ เพื่อขอ verify fact ง่าย ๆ เพิ่ม 2-3 round trips ต่องาน และ latency สูงขึ้น 40% วิเคราะห์แล้วพบว่า 85% ของ verification เป็นแค่ simple lookup — วิธีแก้ที่ได้ผลที่สุดคืออะไร?

- A. เพิ่ม parallelism ของ coordinator ให้ประมวลผล verification request ได้เร็วขึ้น
- B. ให้ synthesis agent ถือ scoped `verify_fact` tool ไว้ทำ simple lookup เอง แล้ว route เฉพาะ verification ที่ซับซ้อนผ่าน coordinator
- C. Cache ผล verification ทั้งหมดไว้ที่ระดับ coordinator เพื่อให้ lookup ซ้ำ ๆ ได้ทันที
- D. ตัด fact verification ออกจาก synthesis workflow ไปเลยเพื่อกำจัด latency ทั้งหมด

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



## เฉลย Exam Sim — ข้อ 1/5

### คำตอบที่ถูก: Option B

scoped cross-role tool จัดการเคส simple ที่เป็น 85% ได้ในตัวเอง กำจัด round-trip latency ส่วนเคสซับซ้อนยัง route ผ่าน coordinator เพื่อจัดการอย่างถูกต้อง

### ทำไมข้ออื่นผิด

A — ประมวลผลเร็วขึ้นไม่ได้กำจัด round trip ที่ไม่จำเป็น ต้นเหตุ latency คือ routing overhead เอง ไม่ใช่ความเร็วของ coordinator

C — cache ช่วยเฉพาะ lookup ช้า แต่ไม่แก้ round-trip overhead ของ verification ครั้งแรกซึ่งเป็นส่วนใหญ่

D — ตัด verification ทั้งทำลายคุณภาพงาน เป้าหมายคือทำให้ verification เร็วขึ้นสำหรับเคสทั่วไป ไม่ใช่ข้ามมันไป



## Exam Sim — ข้อ 2/5 (ลองคิดก่อนดูเฉลย)

document analysis pipeline ต้อง extract metadata ก่อนเสมอ ก่อนจะรัน enrichment tool ใด ๆ — `tool_choice` แบบไหนที่บังคับลำดับนี้ได้?

- A. `tool_choice: { type: "auto" }` — โมเดลจะ extract metadata ก่อนเองโดยธรรมชาติ
- B. `tool_choice: { type: "any" }` — โมเดลต้องเรียก tool แต่จะเลือก `extract_metadata`
- C. `tool_choice: { type: "tool", name: "extract_metadata" }` สำหรับ call แรก แล้วเปลี่ยนเป็น `{ type: "auto" }` สำหรับ call ถัด ๆ ไป
- D. วาง `extract_metadata` ไว้ตัวแรกใน tools array — โมเดลจะเลือก tool ที่อยู่ต้น list ก่อน

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

## ✓ เฉลย Exam Sim — ข้อ 2/5

### คำตอบที่ถูก: Option C

forced selection ( type: "tool" , name: "extract\_metadata" ) การันตีว่าโมเดลเรียก extract\_metadata ก่อน ข้ามหรือสลับลำดับไม่ได้ แล้วเปลี่ยนเป็น auto สำหรับ call ถัดไปเพื่อเลือก enrichment tool ได้ยืดหยุ่น

### ทำไมข้ออื่นผิด

- A — auto ปล่อยให้โมเดลตัดสินใจเองว่าจะเรียก tool หรือไม่ อาจข้าม extract\_metadata ไป enrichment เลย หรือตอบเป็นข้อความ
- B — any การันตีว่าเรียก tool แต่ไม่การันตีว่าตัวไหน โมเดลอาจเลือก enrichment tool แทน extract\_metadata
- D — ลำดับใน array ไม่การันตีลำดับการเลือก โมเดลเลือกจาก description และ context ไม่ใช่ตำแหน่งใน array



## Exam Sim — ข้อ 3/5 (ลองคิดก่อนดูเฉลย)

agent ตัวหนึ่งมี tool ให้เลือกถึง 18 ตัว และเลือก tool ผิดสำหรับ user query อยู่เรื่อย ๆ — สาเหตุรากที่เป็นไปได้มากที่สุดคืออะไร?

- A. tool description ยังไม่ละเอียดพอ ต้องเพิ่มรายละเอียด
- B. agent มี tool มากเกินไป ทำให้ selection reliability ตก — ช่วง optimal คือ 4-5 tools ต่อ agent
- C. โมเดลต้องการ few-shot examples เพื่อเรียนรู้ pattern การเลือก tool ที่ถูกต้อง
- D. ควรตั้ง `tool_choice` เป็น `"any"` เพื่อบังคับให้ใช้ tool

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

## ✓ เฉลย Exam Sim — ข้อ 3/5

### คำตอบที่ถูก: Option B

selection reliability ลดลงเมื่อจำนวน tool เพิ่มขึ้น การให้ agent ถือ 18 tools สร้างความซับซ้อนในการตัดสินใจมากเกินไป ช่วง optimal คือ 4-5 tools ที่ scoped ตามบทบาทเฉพาะของ agent

### ทำไมข้ออื่นผิด

A — description ที่ดีขึ้นช่วยได้ แต่ชดเชยปัญหา tool มากเกินไปไม่ได้ ต่อให้ description สมบูรณ์ 18 tools ก็ยังเยอะเกินไปจะเลือกได้เชื่อก็คือ

C — few-shot examples เพิ่ม token overhead โดยไม่แก้ต้นเหตุ tool overload โมเดลต้องการ tool ที่น้อยและโฟกัสกว่า ไม่ใช่ instruction เพิ่ม

D — "any" บังคับให้เรียก tool แต่ไม่ช่วยเลือกตัวที่ถูกจาก 18 ตัว กลับทำให้แย่ลงเพราะตัดตัวเลือกตอบเป็นข้อความออก



## Exam Sim — ข้อ 4/5 (ลองคิดก่อนดูเฉลย)

subagent ตัวหนึ่งมี `fetch_url` ซึ่งดึง URL จาก domain ใด ๆ ก็ได้ — การปรับปรุงตามหลัก least-privilege ที่ถูกต้องคืออะไร?

A. เพิ่ม URL validation ใน system prompt เพื่อจำกัดว่า agent ควรดึง URL ไหน

B. แบน `fetch_url` ด้วย `load_document` ซึ่งเป็น tool ที่ถูกจำกัด validate เฉพาะ document URL และปฏิเสธ resource ที่ไม่ใช่ document

C. เก็บ `fetch_url` ไว้ แต่เพิ่ม logging เพื่อ monitor ว่า agent ดึงอะไรบ้าง

D. au `fetch_url` ทิ้ง แล้วให้ coordinator จัดการ URL fetching ทั้งหมด

⏸️ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

## ✓ เฉลย Exam Sim — ข้อ 4/5

### คำตอบที่ถูก: Option B

การแทน generic tool ด้วยทางเลือกที่ถูกจำกัดคือการใช้หลัก least privilege ในการออกแบบ tool — `load_document` validate เฉพาะ document URL ป้องกัน misuse และทำให้จุดประสงค์ของ tool ชัดเจนขึ้น

### ทำไมข้ออื่นผิด

- A — instruction ใน system prompt เป็นแค่คำแนะนำ ไม่ได้ถูกบังคับ โมเดลยังเรียก `fetch_url` ด้วย URL ใดก็ได้ ข้อจำกัดควรอยู่ที่ตัว tool เอง
- C — logging ตรวจสอบ misuse หลังเกิดเหตุ แต่ไม่ป้องกัน การป้องกันด้วย constrained tool ดีกว่าการตรวจสอบด้วย monitoring เสมอ
- D — route การ fetch ทั้งหมดผ่าน coordinator เพิ่ม round-trip latency โดยไม่จำเป็นสำหรับงานที่ทำบ่อย constrained local tool มีประสิทธิภาพกว่า



## Exam Sim — ข้อ 5/5 (ลองคิดก่อนดูเฉลย)

extraction pipeline รับ invoice, receipt และ contract โดยไม่รู้ประเภทเอกสารตอน input ต้องการ structured output จาก schema ที่ถูกต้องแบบการ์นตี — ควรใช้ `tool_choice` แบบไหน?

- A. `tool_choice: { type: "auto" }` — โมเดลจะจำเอกสารได้แล้วเรียก tool ที่ถูกต้อง
- B. `tool_choice: { type: "any" }` — โมเดลต้องเรียก tool และจะเลือก extraction schema ที่ถูกต้อง
- C. `tool_choice: { type: "tool", name: "extract_invoice" }` — บังคับใช้ประเภทเอกสารที่พบบ่อยที่สุด
- D. ประมวลผลเอกสารผ่าน extraction tool ทั้งสามตัว แล้วเลือกผลลัพธ์ที่ดีที่สุด

⏸️ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

## ✓ เฉลย Exam Sim — ข้อ 5/5

### คำตอบที่ถูก: Option B

any การันตีว่าโมเดลเรียก tool (ได้ structured output) พร้อมปล่อยให้เลือกเองว่า extraction schema ไหนตรงกับเอกสาร — เป็น use case ที่ any ถูกออกแบบมาพอดี คือการันตี structured output จากหนึ่งในหลาย schema

### ทำไมข้ออื่นผิด

- A — auto ปล่อยให้โมเดลตอบเป็นข้อความแทนการเรียก tool ได้ เมื่อจำเป็นต้องได้ structured output auto ไม่การันตีสิ่งนั้น
- C — บังคับ tool เราจะเจอการเดาประเภทเอกสารที่ยังไม่รู้ tool extract invoice ประมวลผล receipt หรือ contract ให้ถูกไม่ได้
- D — รับทั้งสาม tool เปลือง API call และ context โมเดลระบุประเภทเอกสารและเลือก schema ที่ถูกได้ใน call เดียวด้วย any